Modeling and Analysis of Dependencies between Microservices in DevSecOps

Dr. Jason McZara
Kevla Corporation
Reston, Virginia, USA
jasonmczara@gmail.com

Dr. Subash Kafle
INCOSE WMA
McLean, Virginia, USA
subashke@gmail.com

Dr. Daniel Shin
TIGRIZ Corporation
Hanover, Maryland, USA
daniel.s.shin@hotmail.com

Abstract—Complex microservices-based applications relying on multiple microservices are increasingly commonplace in DevSecOps. These applications are made of microservices that retain their independent ownership, objectives, development, and sustainment approaches. The constituent microservices can have recognized objectives, a designated manager, and resources to provide a desired capability. An outage of a microservices-based application can affect one or more microservices. Although these can be restored or replaced, the ability to cope with this sudden change creates an additional dimension of unpredictability to the overall application. Therefore, rapid and cost-effective recovery of critical applications depends on the ability to identify and focus attention on the most crucial microservices dependencies and interdependencies. The technique used in this research to mathematically model and analyze critical links between microservices relies on an existent effective dependency analysis method called the design structure matrix (DSM. DevSecOps requires quick, repeatable, and efficient identification of critical links between microservices in a microservices-based application. Therefore, the methodology and techniques used in DSM can be leveraged to address the key technical considerations of microservices-based applications in DevSecOps. In this research an outage-impacted microservices-based application is modeled using the proposed methodology and the results are analyzed. Mapping and subsequently prioritizing microservices dependencies and interdependencies directly support the resilience of microservices-based applications.

Keywords—microservices-based application, microservices system of systems, interdependency analysis, unpredictability analysis, application recovery

I. Introduction

Increasingly critical capabilities and services depend on effective microservices-based applications, whose constituents work together to deliver the application capability [1]. In order for the application to provide desired capabilities, the independent microservices must be coordinated. An example of a ride-sharing application that depends on various internal and external microservices is given in Fig. 1.

As seen in Fig. 1, various independent microservices such as payments microservice, driver service, and notifications service are linked to serve the common goal of providing a single user interface (UI) application with ride-sharing functionality. These microservices can have recognized objectives and a designated manager to provide a desired capability.

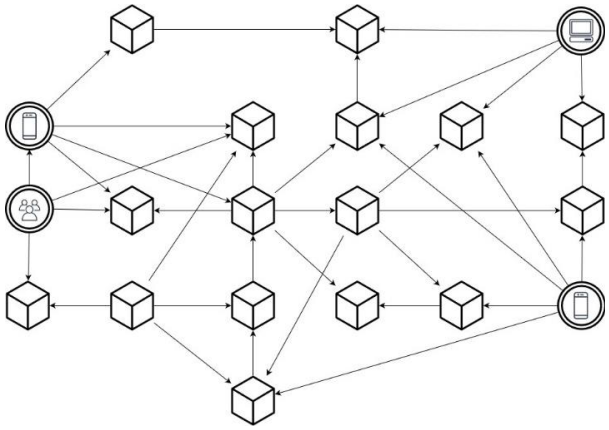

Fig. 1. An example of a microservices-based application

The increased reliance on microservices-based applications is evident in all commercial and public sectors. In such an application the flow of information between the constituent microservices is key to the successful implementation and delivery of the application capability. For example, the information flow between the microservices via application programming interfaces (APIs) for the microservices-based application in Fig. 1 is critical for the successful execution of the application's ride-sharing capability. However, it is challenging to identify the critical information links between these microservices, their integration, and testing. Therefore, the key to the successful delivery of the capability in a microservices-based application depends on managing these information links and interdependencies between the constituent microservices. In addition, the coordination between the application level and microservice levels is equally important.

In some microservices-based applications, the constituent microservices retain their independent ownership, objectives, funding, development, and sustainment approaches, while the application has recognized objectives, a designate manager, and resources to provide a desired capability [2, 3]. The ownership, emergence, abstractness, and stability of states are some of the major challenges of microservices-based applications [4]. Because these are not always designed as one large application in one development cycle, there is no single application owner; hence, it is difficult to establish the ownership of the interfaces between the constituent microservices [4].

An outage can affect one or more constituent microservices. Although the affected microservice can be restored or replaced,

978-1-7281-6547-9/20/$31.00 ©2020 IEEE
DOI 10.1109/SmartCloud49737.2020.00034

the ability to cope with this sudden change creates an additional dimension of unpredictability to the microservices-based application. Indeed, during a microservice failure or outage, the affected microservices must be quickly recovered in order to restore the application capability. However, during this recovery time, it is difficult to effectively integrate and test new or replacement microservices, and to identify the dependencies between the restored microservices and the existing ones. Therefore, identifying which dependencies to restore and test is critical to the recovery time of the capability or service provided by the microservices-based application [5].

In many aspects there are similarities between component based systems (CBS) and microservices-based applications. Both follow a modular design methodology where independent, fully functional elements are integrated into a larger structure to deliver unique capabilities [6]. The key to understanding any microservices, be it CBS or microservices-based application, is understanding the relationships and interfaces between the microservices' elements [4].

This research, which uses a proven approach to identify and prioritize the microservices dependencies and interdependencies in a microservices-based application, is applied to the DevSecOps domain to mathematically model and identify the critical interfaces between the constituent microservices. The mathematical modeling and identification of the critical links between the constituent microservices is done using an existent dependency analysis method, the design structure matrix (DSM). A DSM is defined as a "simple, compact and visual representation of a complex" application [7]. Also commonly called dependency structure matrix [8], the DSM has helped analyze the interrelationships between activities involved in application design [9, 8].

Critical microservices-based applications are evolving with or without explicit intent, and as part of an DevSecOps environment, are vulnerable to unexpected outages during their development. One way to address this threat to a microservices-based application is to understand, identify, prioritize, and mitigate the relationships and interface vulnerabilities during the DevSecOps process [10, 11, 12, 4]. In this research, an outage-impacted microservice is used to model a microservices-based application. The preliminary results of the mathematical modeling and analysis indicate that it is possible to identify and prioritize the key integration points during microservice recovery, reducing the unpredictability of sudden changes and improving the recovery time in case of an outage [5, 13].

Mapping and subsequently prioritizing microservices dependencies directly supports the resilience and recovery of a microservices-based application. Uncovering and analyzing direct or indirect microservices dependencies allow the development and implementation of application recovery solutions and plans for the microservices-based application. The analysis proposed in this paper contributes to bridging the gaps in existing microservices-based application recovery and microservices interdependency analysis. The identification of critical microservices dependencies and their interdependencies as part of the application recovery plan using the proposed analysis contributes to rapid application recovery during an outage. In this research an outage-impacted microservices-based application is modeled using the proposed methodology and the results are analyzed. Mapping and subsequently prioritizing microservices dependencies and interdependencies directly support the resilience of microservices-based applications.

II. MICROSERVICES-BASED APPLICATION

When applying DevSecOps to microservices-based applications, one of the biggest challenges is addressing the dependencies and interdependencies between the constituent microservices and their impact on the application. How the data will flow and how to control and manage this flow are key aspects of microservices dependency analysis [14]. The term microservices-based application covers a wide range of capabilities delivered through constituent services. Table 1 displays the different types of microservices-based applications.

TABLE 1. TYPE OF MICROSERVICES-BASED APPLICATIONS

Microservices-Based Application Types	Description
Directed	Microservices-based application objectives, management, funding, and authority; microservices are subordinated to microservices-based applications
Acknowledged	Microservices-based application objectives, management, funding, and authority; some microservices DevSecOps can be under their own management, funding, and authority in parallel with the microservices-based application
Collaborative	No top-down objectives, management, authority, responsibility, or funding at the microservices-based application level; microservices voluntarily work together to address shared or common interests and might be externally developed and managed
Virtual	Similar to collaborative, but the microservices do not know about each other

An engineering and design consideration of an acknowledged microservices-based application (as defined in Table 1) is enabling the flow of data between the constituents and ensuring functionality across the application while balancing the needs of the constituent services. Examples of such applications include those that leverage external microservices. Although some of the constituent microservices can independently provide API responses, the multi-layered structure of the application is only achieved through the constituent microservices working together.

III. BUSINESS CONTINUITY AND APPLICATION RECOVERY

The events posing business continuity risks can be caused by humans with malicious intent or by accident, by natural events such as hurricanes or earthquakes, or by random hardware or software malfunctioning. Although every category requires consideration of different solutions for business continuity, the extent of the impact of the disaster in a microservices-based application context depends on how many constituent microservices become unavailable [15, 16].

In addition to the need to address recovery requirements as part of DevSecOps, customer needs and the critically of applications have increasingly required that microservices

offered through microservices-based application settings address the needs for recovery and business continuity. Disaster recovery and business continuity are key areas that need to be considered by DevSecOps teams, and capabilities that are enabled by constituent microservices in a microservices-based application setting [3].

Some of the key characteristics of a microservices-based application are the managerial independence of the component microservices, along with the distribution of the microservices. Therefore, in case of an outage, one or more constituent microservices could be affected while the rest are unaffected. The microservices that become unavailable directly affect the capability and functionality provided by the microservices-based application. Therefore, in order to restore the application capability during a microservice failure or outage, it is key to quickly restore the affected microservices. A highly resilient application is not only one less likely to fail but also one that is able to recover promptly from the outage state [17].

Table 2 defines the four main states of a microservices-based application after an outage caused by the failure of one or more of its constituent microservices.

TABLE 2. THE STATE OF RECOVERY IN THE CONTEXT OF A MICROSERVICES-BASED APPLICATION

State	Description SoS State
Emergency Response	Major microservices-based application disruption detected Activate contingency plans Stabilize affected areas
Resume	Restart limited microservices-based application capability in affected areas
Recover	Restart full microservices-based application capability in affected areas Curtail service performance levels
Restore	Restart full application performance and availability Collect lessons learned

However, during this recovery time, it is difficult to integrate and test new or replacement microservices [18] and to identify the dependencies between the restored microservices and existing microservices. From a testing and evaluation perspective, identifying which links to restore and test poses major challenges to the recovery time of the capability provided by the microservices-based application. As seen in Fig. 2, by prioritizing the links and subsequently restoring those with the highest priority, the recovery stage can be minimized and the microservices-based application can be restored.

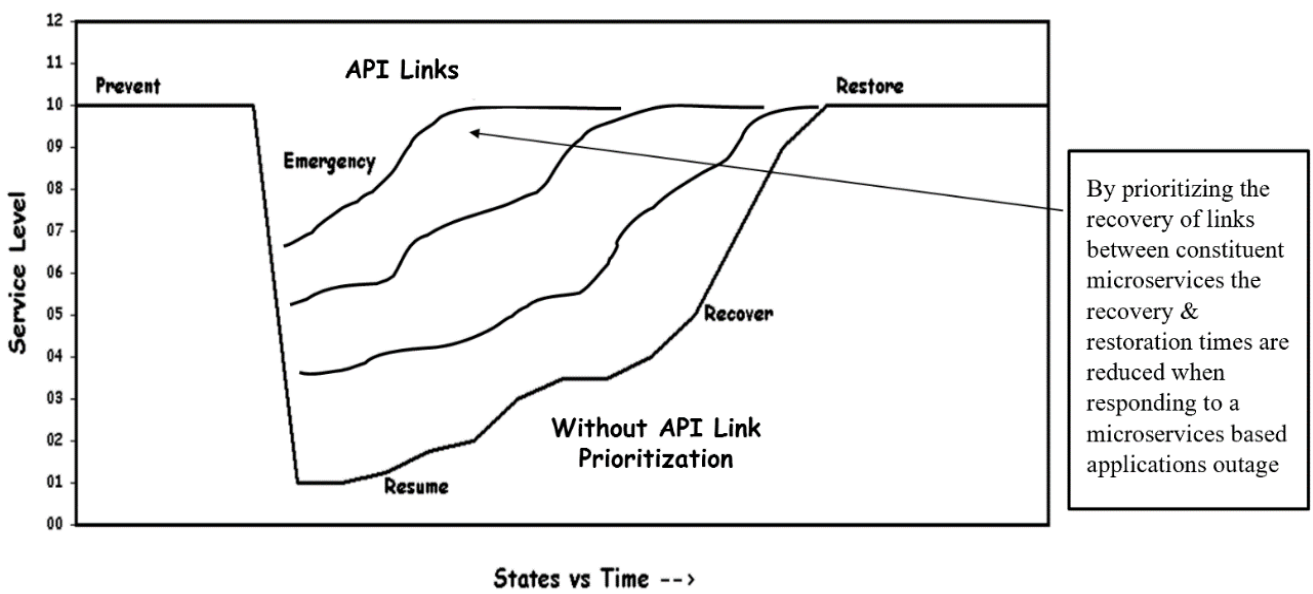

Fig. 2. Microservices-based application state vs. time after an emergency. The recovery time is reduced by prioritizing the links for recovery.

IV. Approach

In order to address disaster recovery in the context of a microservices-based application it is important to investigate the DevSecOps elements of the application (Fig. 3). The DevSecOps methodology provides a framework for creating and maintaining microservices-based applications [11]. The core DevSecOps elements include plan, code, build, test, release, deploy, operate, monitor, and plan. Desgining security features should be considered at every phase of the DevSecOps process [12]. Frequently code and build, release and deploy, and operate and monitor are paired together for simplicity.

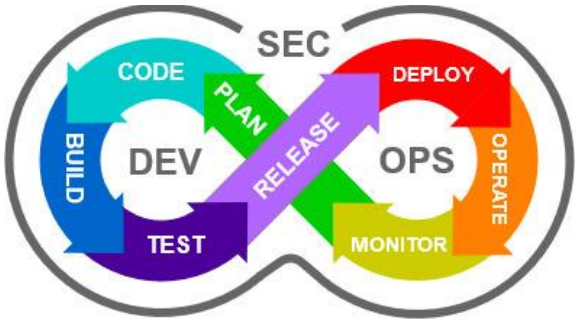

Fig. 3. Core DevSecOps phases and elements

For the successful recovery of microservices-based application capabilities, early recovery planning and microservices resilience engineering is vital and cannot be an afterthought [12]. As outlined in Table 3, it is beneficial to make various recovery considerations for a microservices-based application during the completion of the DevSecOps phases.

TABLE 3. DISASTER RECOVERY CONSIDERATIONS THAT SHOULD BE MADE AS PART OF DEVSECOPS FOR MICROSERVICES-BASED APPLICATIONS

Core DevSecOps Phases and Elements	Recovery Considerations for Microservices-Based Applications
Plan	Performing a business impact analysis at the microservices-based application level
Code and Build	Capturing microservices dependencies and links
Test	Assessing the effect of single or multiple microservices failure and the effect it will have on the application capability
Release and Deploy	Creating redundant links or standardization of links for rapid recovery of microservices-based application capability
Operate and Monitor	Ensuring the constituent microservices focus on the recovery capability of their microservice while considering the dependencies with other microservices
Plan	Analyzing the effect of the upgrade on links between constituent microservices

As microservices-based applications grow larger and more complex, modeling and computing the interactions within and between microservices becomes challenging. This poses a critical risk to the resilience of a microservices-based application [16, 19, 17]. Similarly, [20] suggest that in microservices-based application environments, the recovery and risk management is challenging since top-down risk management is not possible in such environments [16].

Managing dependency and impact analysis during a microservices failure or upgrade using traditional processes such as a component dependency matrix is insufficient because this matrix analyzes only the relationship between the constituent microservices but not between these and interfaces [21, 22]. Prioritizing the links is possible only with a method that distinguishes microservices links based on their interdependencies [23, 13].

As already noted, microservices-based applications are characterized by dependencies between their constituent microservices. These dependencies give rise to microservices' "co-evolution" (i.e., changing one microservice results in changes required to the linking microservices) [23]. The higher the number of dependent microservices in a complex microservices-based application, the higher the microservice co-evolution will be. Therefore, the recovery of the constituent microservices depends on the microservices' owner. In an acknowledged microservices-based application, its recovery is dependent on the actions of the constituent microservices. As part of the application agreement, the microservices' owners are responsible for designing them to meet the availability recovery time objectives, and the application manager focuses on the links between the constituent microservices.

Managing the integration points and links between the constituent microservices is difficult; therefore, this research focuses on the interdependencies between the constituents of a microservices-based application.

V. Mathematical Modeling And Simulation

Mathematical modeling and simulation of microservices-based applications has become a valuable tool for interdependency analysis within such applications [24]. A comprehensive, automated, and maintainable modeling methodology can be very useful for the successful integration testing and functional testing of microservices-based applications. Failure to conduct adequate application integration modeling as part of DevSecOps can lead to potentially catastrophic failures after the recovery of the constituent microservices. A three-step approach is proposed based on [23]'s multi-dimensional dependency (MDD) analysis model. The model (Fig. 4) was proposed to identify and prioritize critical dependencies between microservices.

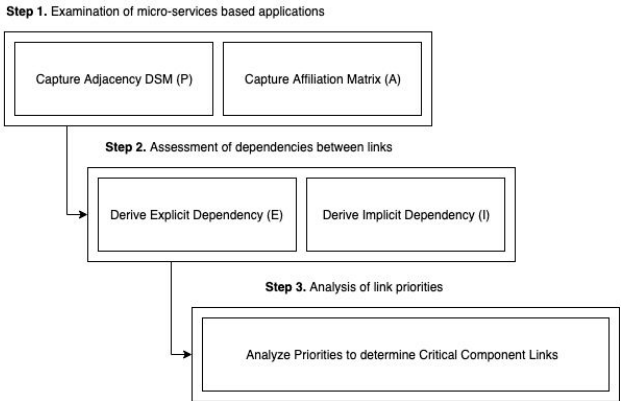

Fig. 4. Proposed method to prioritize the links between microservices in an acknowledged microservices-based application affected by an outage [23]

In this research the illustration is carried out from the perspective of one of the microservice owners (Microservice 1) of an example of a microservices-based application architecture (Fig. 5).

Step 1. Examination of Microservices-Based Application Architecture

The DSM is applicable to capture the critical interfaces between the microservices of a microservices-based application [25]. In the example in Fig. 5 the first stage would be to examine the links between the constituent microservices.

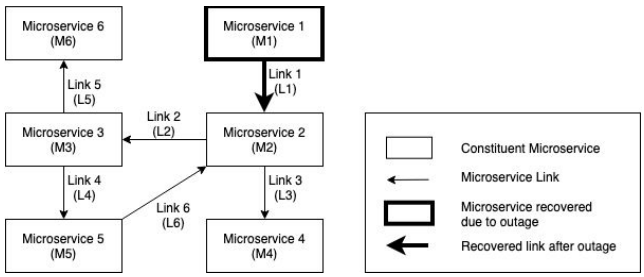

Fig. 5. Example of a microservices-based application with Microservices 1 in recovery configuration due to outage [23]

Microservices links are captured using standard DSM processes. The matrix in Fig. 6 identifies how the microservices are interlinked and captures the information exchanged between them. The information on the microservices and their links is identified using microservices-based application architecture diagrams and other related artifacts.

	M1	M2	M3	M4	M5	M6
M1	0	1	0	0	0	0
M2	0	0	1	1	0	0
M3	0	0	0	0	1	1
M4	0	0	0	0	0	0
M5	0	1	0	0	0	0
M6	0	0	0	0	0	0

Fig. 6. Microservices vs. microservices martix; captured based on DSM (P)

As outlined in Fig. 7, a recovered microservice might be set up differently than the original microservice impacted by the outage. Hence, the link between the recovered microservice and other constituent systems can be affected or changed. Therefore, when a microservice is recovered, it is important to investigate the links between it and the other microservices to determine which links are affected and which links should be recovered first.

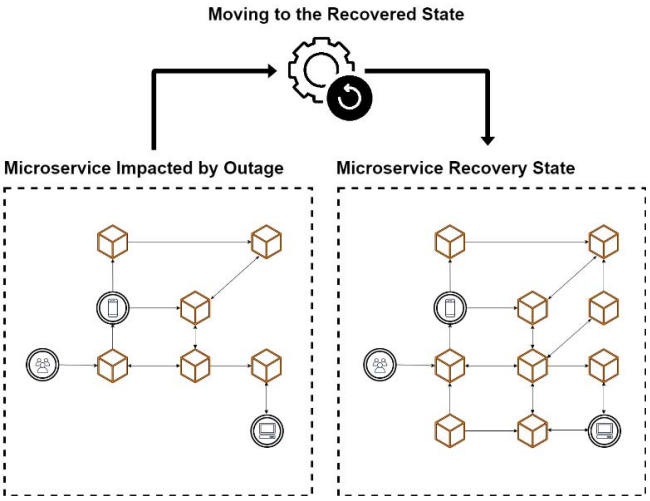

Fig. 7. The recovered microservice has a different architecture and interface than the original microservice impacted by the disaster.

The microservices-based application affiliation matrix is completed also by capturing microservices versus links. This matrix (Fig. 8) identifies and captures the type and level of information exchange through a microservice's link.

	M1	M2	M3	M4	M5	M6
L1	0	1	0	0	0	0
L2	0	0	1	0	0	0
L3	0	0	0	1	0	0
L4	0	0	0	0	1	0
L5	0	0	0	0	0	1
L6	0	1	0	0	0	0

Fig. 8. System vs. link matrix (A)

There could be many types of links, as the microservices interact, exchange information, and communicate with one another. Although in this example the links are assumed as uniform, if different types of links exist, then separate matrices need to be developed based on each type of link. Also, the level of information exchanged between the constituent microservices can be captured by a range of values as opposed to the binary values (0 and 1).

Step 2. Investigation of Dependencies between API Links

It is important to prioritize the API links that need to be recovered. However, this prioritization is only possible if the links can be distinguished based on their dependencies. During the recovery stage both time and human resources are limited; therefore, it is necessary to recover and test the minimum number of integration links while satisfying the microservices-based application capability and meeting the risk management requirements.

Within a microservices-based application, two microservices not directly connected may depend on data and information from each other. This means the information exchange via the integration links can be direct or indirect. A direct link connects two microservices; for example, in Fig. 9,

both link i and link j are connected to Microservice k, and there is a dependency between these two links. This dependency is defined as explicit dependency and is captured by using the operation of microservice vs. link matrix shown in Fig. 8.

Fig. 9. Explicit dependency between link i link j as information flows through Microservice k [22]

The following equations are used to calculate the microservice explicit matrix (E) [22]:

$$\boldsymbol{E} = \boldsymbol{A} \times \boldsymbol{A^T} \text{ and} \tag{1}$$

$$\boldsymbol{e_{ij}} = \sum_{\boldsymbol{i=1}}^{\boldsymbol{m}} \boldsymbol{a_{ik} a_{jk}}, \tag{2}$$

where A^T is the transpose of A; $E_{n,n}$ is a square, symmetric matrix with n rows and n columns; and e_{ij} in matrix E represents an off-diagonal cell.

The calculated E matrix for the microservices-based application example is shown in Fig. 10.

	L1	L2	L3	L4	L5	L6
L1		0	0	0	0	1
L2	0		0	0	0	0
L3	0	0		0	0	0
L4	0	0	0		0	0
L5	0	0	0	0		0
L6	1	0	0	0	0	

Fig. 10. Explicit dependency matrix for the microservices-based application example

Information exchange via the integration links can create an indirect dependency between the links. This indirect dependency is important in prioritizing the links during recovery. As seen in Fig. 11, there exists a link between the affiliated microservices and their respective neighboring microservices; link j is indirectly dependent on link i.

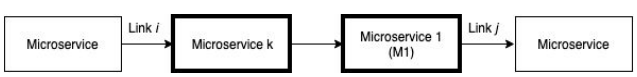

Fig. 11. Implicit dependency between link i and link j as information flows through Microservice k and Microservice l [23]

The DSM matrix and the microservice affiliation matrix are used to capture the implicit dependencies (I) through the following equations [23]:

$$\boldsymbol{I} = \boldsymbol{A} \times \boldsymbol{P} \times \boldsymbol{A^T} \quad \text{and} \tag{3}$$

$$\boldsymbol{I} = \sum_{\boldsymbol{\lambda=1}}^{\boldsymbol{n}} \boldsymbol{A} \times \boldsymbol{P^{\lambda}_{actual}} \times \boldsymbol{A^T}, \tag{4}$$

where n is the number of total components of the microservice and $\boldsymbol{P^{\lambda}_{actual}}$ is the DSM capturing the number of instances a microservice is used during the information flow between microservices that are λ links apart. Fig. 12 displays the calculated I matrix for the microservices-based application example.

	L1	L2	L3	L4	L5	L6
L1		12	4	11	2	7
L2	12		4	16	4	12
L3	0	0		0	0	0
L4	16	12	4		2	16
L5	0	0	0	0		0
L6	7	12	4	11	2	

Fig. 12. Implicit dependency matrix for the microservices-based application example

At this stage the direct and indirect dependencies are transitioned into dependency strength between microservice links; these are not dependencies between microservices but rather dependencies between links. The strength of the link priorities is obtained through the addition of the implicit and explicit dependencies resulting in the total link dependency strength matrix (LDM) shown in Fig. 13.

	L1	L2	L3	L4	L5	L6
L1		12	4	11	2	8
L2	12		4	16	4	12
L3	0	0		0	0	0
L4	16	12	4		2	16
L5	0	0	0	0		0
L6	8	12	4	11	2	

Fig. 13. Total link dependency matrix for the microservices-based application example

Step 3. Analysis of API Link Priorities

The final stage involves analyzing the dependency strength matrix and identifying API link priorities. The dependency strength is directly proportional to link priority. For the example provided, Fig. 14 shows the prioritized links that have to be recovered and tested should Microservice 1 be affected by an outage. Furthermore, if the goal is to determine the priority of links that are dependent on L1, by looking at the highlighted row in Fig. 13 and translating the dependency strength into priorities, it can be seen that L2 has the highest dependency strength value; hence, L2 has the highest priority link with respect to L1. Similarly, L5 has the lowest dependency strength value; hence, it has the lowest testing priority.

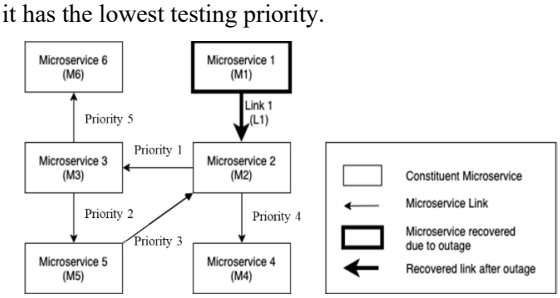

Fig. 14. Prioritization of link recovery and testing in a microservices-based application assuming Microservice 1 is recovered or upgraded following a disaster

VI. Summary And Conclusions

DevSecOps for microservices-based applications is different from traditional application development. The former poses

particular challenges due to evolving requirements, asynchronous design, development and integration, and the existence of multiple stakeholders. Recovery planning for a microservices-based application would benefit if focus was placed on the dependencies between the constituent microservices.

The recovery of a microservices-based application is simplified through the prioritization of integration links. This prioritization enables critical links to be located while valuable and scarce resources during the recovery stage are focused on the recovery and integration testing of the high-priority links.

The analysis of the dependency between microservice links involves investigating the microservice-to-microservice relationship and microservice-to-link and link-to-link relationships. By using the DSM and modified organizational communication and relationship analysis techniques, it is possible to identify the dependency between links. Hence, it is possible to calculate their relative priority based on the strength of their dependency. It is demonstrated that applying the MDD method and the effective and proven dependency analysis technique, it is possible to identify the highest-priority link in a microservices-based application by calculating the dependency strengths between microservices links.

As part of the continued research, microservices and their recovery stage are modeled using the proposed approach. The preliminary results of the mathematical modeling and analysis indicate that the key integration points during microservice recovery can be identified and prioritized. The identification of critical microservices dependencies and their interdependencies as part of the recovery plan using the proposed analysis contributes to rapid application capability recovery during a major outage. These findings demonstrate that applying the MDD method to the domain of microservices-based application recovery is a promising and feasible approach that can be integrated into DevSecOps and warrants additional empirical studies. Future research should be focused on the modeling and simulation of an actual microservices-based application.

REFERENCES

[1] C. Ncube, S. L. Lim, and H. Dogan, "Identifying Top Challenges for International Research on Requirements Engineering for Systems of Systems Engineering" in the *21st IEEE International Requirements Engineering Conference* 2013, IEEE, 2013, pp. 342–344.

[2] Systems Engineering Body of Knowledge, *System of Systems – Application of SE to System of Systems*, 2016, Retrieved from http://sebokwiki.org/wiki/Systems_of_Systems_(SoS)

[3] M. Anisetti, C. Ardagna, E. Damiani, and F. Gudenzi, *"A Continuous Certification Methodology for DevOps," MEDES*, 2019.

[4] M. A. Wilson, "Visualizing Critical Infrasttucture as a System of Systems," in *SEDC*, Chantilly, VA, 2016.

[5] D. S. Cruzes, M. G. Jaatun, and T. D. Oyetoyan, "Challenges and Approaches of Performing Canonical Action Research in Software Security," *HotSoS*, 2018.

[6] Department of Defense, *Systems Engineering Guide for Systems of Systems*, Office of the Under Secretary of Defense, 2008.

[7] T. R. Browning, "Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions," *IEEE Transactions on Engineering Management,* 48(3), 2001, pp. 292–306.

[8] I. M. Srour, M. A. U. Abdul-Malak, A. A. Yassine, and M. Ramadan, "A Methodology for Scheduling Overlapped Design Activities Based on Dependency Information," *Automation in Construction* 29, 2013, pp. 1–11.

[9] D. V. Steward, "Partitioning and Tearing Systems of Equations," *Journal of the Society for Industrial & Applied Mathematics, Series B: Numerical Analysis 2*, 1965, pp. 345–365.

[10] D. S. Cruzes, M. G. Jaatun, and K. Rindell, "The Security Intention Meeting Series as a Way to Increase Visibility of Software Security Decisions in Agile Development Projects," *ARES*, 2019.

[11] N. Elissawy, R. Moghaddam, J. Park, K. Sharma, and H. Singh, *Composable DevSecOps Architecture: The Need for Secure and Flexible Deployment*, EasyChair Preprint, 2020.

[12] B. S. Rahul, K. Prajwal, and M. N. Manu, "Implementation of DevSecOps using Open-Source tools," *International Journal of Advance Research, Ideas and Innovations in Technology* 5(3), 2019.

[13] S. B. O. Gennari Carturan and D. H. Goya, "A Systems-of-Systems Security Framework for Requirements Definition in Cloud Environment," *ECSA*, September 9–13, 2019.

[14] R. Herardian, "The Soft Underbelly of Cloud Security," *IEEE Computer and Reliability Societies*, May/June 2019.

[15] M. L. Ali, B. Atobatele, and K. Thakur, "Challenges of Cyber Security and the Emerging Trends," *BSCI*, July 2019.

[16] P. Gates-Idem, M. Lavin, and E. Zheng, "Building a Virtually Air-gapped Secure Environment in AWS with Principles of DevOps Security Program and Secure Software Delivery," *HotSoS*, April 2018.

[17] P. Uday, P. and K. Marais, "Designing Resilient Systems-of-Systems: A Survey of Metrics, Methods, and Challenges," *Systems Engineering* 18(5), 2015, pp. 491–510.

[18] D. Bodeau, J. Brtis, R. Graubart, and J. Salwen, "Resiliency Techniques for Systems-of-Systems Extending and Applying the Cyber Resiliency Engineering Framework to the Space Domain," in the *7th International Symposium on Resilient Control Systems (ISRCS),* IEEE, 2014, pp. 1–6.

[19] A. Putta, "Scaling Agile Software Development to Large and Globally Distributed Large-scale Organizations," *ACM/IEEE 13th International Conference on Global Software Engineering*, 2018.

[20] A. Cavallo, and V. Ireland, "Preparing for Complex Interdependent Risks: A System of Systems Approach to Building Disaster Resilience," *International journal of disaster risk reduction*, *9*, 2014, pp. 181–193.

[21] M. Larsson, *Applying Configuration Management Techniques to Component-Based Systems* (Dissertation), Uppsala, Sweden: Uppsala University Department of Information Technology, 2000.

[22] S. M. Huang, C. F. Tsai, and P. C. Huang, "Component-based Software Version Management Based on a Component-Interface Dependency Matrix," *Journal of Systems and Software* 82(3), 2009, pp. 382–399.

[23] S. Kafle, S. Sarkani, and T. A. Mazzuchi, "Prioritization of Component-to-Component Links for Testing in Component Based Systems," *International Testing and Evaluation Association*, 2015.

[24] M. Ouyang, "Review on Modeling and Simulation of Interdependent Critical Infrastructure Systems," *Reliability Engineering & System safety* 121, 2014, pp. 43–60.

[25] Sosa, Manuel E., "A Structured Approach to Predicting and Managing Technical Interactions in Software Development," *Research in Engineering Design* 19(1), 47–70, 2008.

AUTHORS' BIOGRAPHY

Dr. Jason McZara is a lead cloud engineer and architect working in the areas of cloud architecture and development, migration, operations, DevSecOps, as well as IT systems engineering, IT project management, and IT service management. His research interests include DevOps container services, cloud security, and multi-cloud interoperability and interdependency. Dr. McZara holds several expert and professional cloud vendor certifications including Microsoft Azure and Amazon Web Services. He received a Ph.D. in

Systems Engineering from the George Washington University and an M.S. and a B.S. in Engineering from Virginia Tech.

Dr. Subash Kafle is a lead systems engineer supporting multiple government agencies in their modernization efforts. He holds a Ph.D. in Systems Engineering from the George Washington University and an M.S. and B.S. from Virginia Tech. Dr. Kafle is an Agile Certified Scrum Master and is professionally certified in Amazon Web Services, Software Testing, ITIL, and IT Service Management. He served as a president for the INCOSE WMA chapter, and he is passionate about supporting STEM related programs by mentoring future systems engineers.

Dr. Daniel Shin is a principal systems engineer at TIGRIZ Corporation, supporting the United States Federal Government Agencies and Commercial IT Businesses. He holds a Ph.D. from the Department of Engineering Management and Systems of the George Washington University, an M.S. in Systems Engineering from the George Washington University, and a B.S. in Aerospace Engineering from Virginia Tech. Dr. Shin is an expert in systems and database architecting, cloud engineering, IT infrastructure, integration, and operation and maintenance. His areas of expertise include the development process of complex engineering design, from concept to implementation with agile and DevSecOps approaches. Dr. Shin received a Level III certification in Science and Technology Management from the Defense Acquisition University.